

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа 4

Выполнил:

Алексеев Николай

Группа К3344

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1 Задача

Задание:

1. Разделите ваше монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service
2. Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом
3. Спроектируйте разделение вашей БД на обособленные БД
4. Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
5. Опишите варианты запросов и ответов, задокументируйте возможные ошибки
6. В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения вашего монолитного бэкенд-приложения на отдельные микросервисы

2 Ход работы

Для разделения монолита на микросервисы решил выделить основные сущности, получилось 3:

- Юзер
- Рецепты
- Взаимодействия

В сервис Юзер входят:

- Управление пользователем
- Подписки
- Созданные рецепты
- Регистрация/авторизация
- JWT

В рецепты входят:

- CRUD рецептов
- Шаги
- Ингредиенты
- Поиск

- Фильтрация
- Публикация

Во взаимодействия входят:

- Лайки/дизлайки
- Сохранения
- Комментарии
- Счетчики взаимодействий

В монолитной БД 11 таблиц:

- Comments
- Cuisines
- Ingredients
- Likes
- Dislikes
- Recipes
- Saved_recipes
- Steps
- Subscriptions
- Type_recipes
- Users

Для каждого микросервиса своя база данных.

Юзер:

- Users
- subscriptions

Рецепт:

- recipes
- steps
- ingredients
- cuisines
- type_recipes

Взаимодействия:

- likes
- dislikes
- saved_recipes
- comments

Сервисы напрямую обращаются только к своим БД, к чужим БД через API.

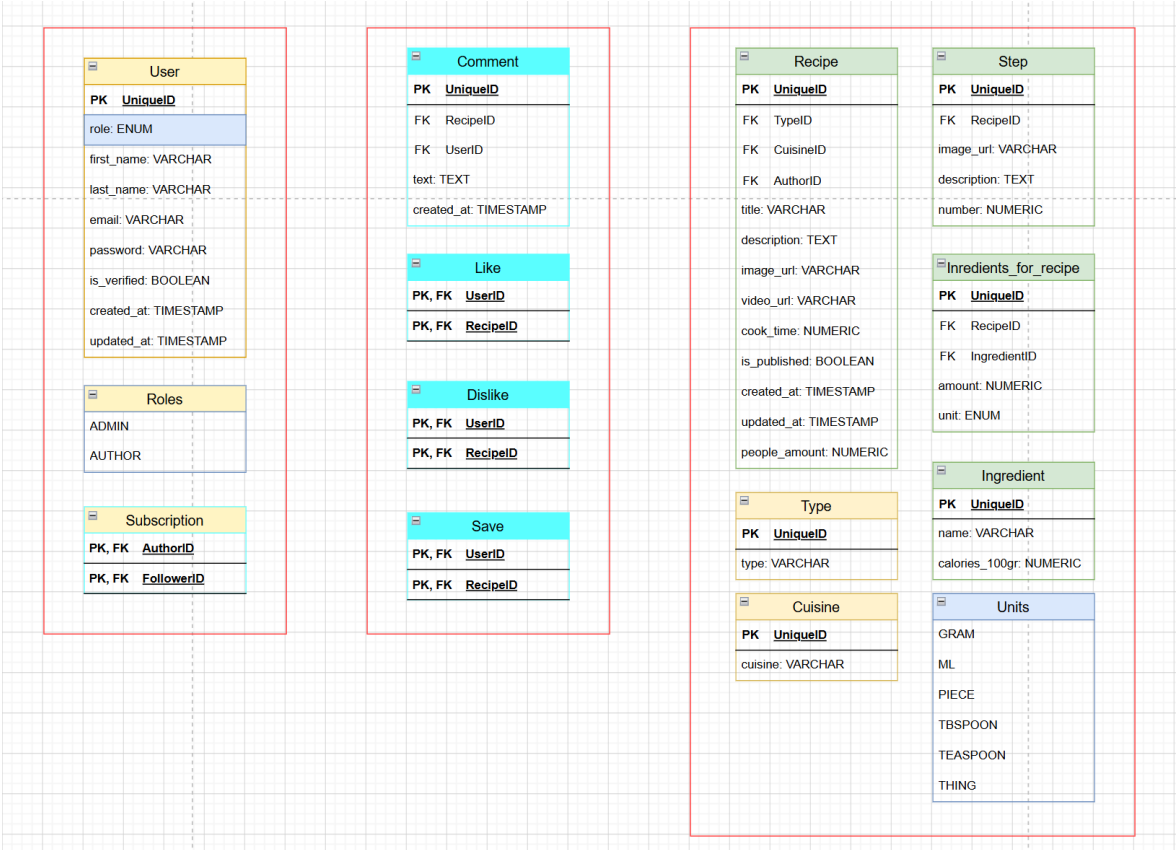


Рисунок 1 – БД разделенная на сервисы

Метод	Путь	Сервис	Описание
POST	/api/auth/register	Юзер	Регистрация нового пользователя
POST	/api/auth/login	Юзер	Аутентификация пользователя
POST	/api/auth/logout	Юзер	Выход из системы
POST	/api/auth/refresh	Юзер	Обновление JWT токена
GET	/api/users/me	Юзер	Получение профиля текущего пользователя
PATCH	/api/users/me	Юзер	Обновление профиля текущего пользователя
GET	/api/users/:id	Юзер	Получение профиля пользователя по ID
GET	/api/users/me/subscriptions	Юзер	Получение подписок текущего пользователя

POST	/api/users/:id/subscribe	Юзер	Подписка на автора
DELETE	/api/users/:id/subscribe	Юзер	Отписка от автора
GET	/api/users/me/recipes	Рецепт	Получение рецептов текущего пользователя
GET	/api/users/:id/recipes	Рецепт	Получение рецептов пользователя по ID
GET	/api/recipes	Рецепт	Список рецептов (пагинация, фильтрация)
GET	/api/recipes/:id	Рецепт	Получение полной информации о рецепте
POST	/api/recipes	Рецепт	Создание нового рецепта
PATCH	/api/recipes/:id	Рецепт	Обновление рецепта
DELETE	/api/recipes/:id	Рецепт	Удаление рецепта
POST	/api/recipes/:id/publish	Рецепт	Публикация рецепта
GET	/api/search	Рецепт	Поиск рецептов по ключевым словам
GET	/api/users/me/saved-recipes	Взаимодействие	Сохраненные рецепты пользователя
GET	/api/users/me/liked-recipes	Взаимодействие	Лайкнутые рецепты пользователя
GET	/api/users/me/disliked-recipes	Взаимодействие	Дизлайкнутые рецепты пользователя
POST	/api/recipes/:id/like	Взаимодействие	Поставить лайк
DELETE	/api/recipes/:id/like	Взаимодействие	Убрать лайк
POST	/api/recipes/:id/dislike	Взаимодействие	Поставить дизлайк
DELETE	/api/recipes/:id/dislike	Взаимодействие	Убрать дизлайк
POST	/api/recipes/:id/save	Взаимодействие	Сохранить рецепт
DELETE	/api/recipes/:id/save	Взаимодействие	Удалить из сохраненных
GET	/api/recipes/:id/comments	Взаимодействие	Получить комментарии рецепта
POST	/api/recipes/:id/comments	Взаимодействие	Добавить комментарий
DELETE	/api/comments/:id	Взаимодействие	Удалить комментарий

Эндпоинты для мерсервисного взаимодействия:

GET /internal/users/{id}

Ответ:

```
{
  "id": 123,
  "login": "john_doe",
  ...
}
```

```
}
```

POST /internal/users/batch

Тело:

```
{"userIds": [123, 456, ...]}
```

```
[
```

```
{
```

```
  "id": 123,
```

```
  "login": "john_doe",
```

```
  ...
```

```
},
```

```
{
```

```
  "id": 456,
```

```
  "login": "jane_smith",
```

```
  ...
```

```
},
```

```
...
```

```
]
```

GET /internal/recipes/{id}

Ответ:

```
{
```

```
  "id": 456,
```

```
  "title": "Борщ классический",
```

```
  ...
```

```
}
```

POST /internal/recipes/validate (для проверки существования рецептов)

Тело:

```
{
```

```
  "recipeIds": [456, 789, 999]
```

```
}
```

Ответ:

```
{
```

```
  "existing": [456, 789],
```

```
"missing": [999]
}
```

GET /internal/recipes/{id}/counters

Ответ:

```
{
  "likes": 42,
  "dislikes": 3,
  "comments": 15,
  "saves": 10
}
```

Возможные ошибки

400	Некорректные параметры
401	Отсутствует или недействителен токен
403	Недостаточно прав для операции
404	Запрашиваемый ресурс не существует
409	Конфликт (дубликат)
422	Данные не прошли валидацию
500	Внутренняя ошибка сервиса
503	Сервис не отвечает

Для перехода на микросервисы надо:

- Подготовить инфраструктуру Docker, RabbitMQ и т.д.
- Выделить модули в отдельные сервисы
- Разделить БД
- Реализовать межсервисное взаимодействие
- Переключить маршруты
- Убрать старый код

3 Вывод

В результате выполнения лабораторной работы мной был проанализирован монолитный сервис, из него было выделено 3 модуля для создания 3 отдельных микросервисов. Была продумана реализация раздельных баз данных с межсервисным взаимодействием,

предусмотрена обработка разнообразных ошибок, спроектированы внутренние
эндпоинты. Это должно упростить поддержку кода, масштабирование проекта.